

EMBEDDED PS

ESP32-Based Terminal with NTP Timestamping, Offline Queueing, and Emergency Override

Report

Submitted by: Yash Raj (250102107)

1. Introduction

This report explains how a simple keypad door-lock system was built and tested using an ESP32 microcontroller, simulated on the Wokwi platform. The basic idea is a normal PIN-entry lock, but with three extra features added on top: it sends a record of every access attempt to the cloud, it keeps working even when the WiFi is unreliable, and it has a physical emergency button to let someone in if needed.

To use it, someone types a 4-digit PIN on a keypad. The system checks the PIN against a small list of allowed codes and either grants or denies access, showing the result on a small LCD screen and switching on two outputs that stand in for a real lock or relay. Every attempt, whether it succeeds or fails, gets a timestamp and is sent to a server as a small JSON message. If the WiFi happens to be down when this happens, the log isn't lost — it's saved locally and sent automatically as soon as the connection comes back

2. Objective

GOAL : Purely local access-control state machine and extend it to meet the following functional requirements:

- Transform the local scan data into a JSON payload using the ArduinoJson library
- Connect to the Wokwi-GUEST Wi-Fi network. Fetch accurate timestamps via an NTP server to attach to each transaction.
- Identify network blackouts. Create a robust fallback mechanism (an offline array/queue) to save up to 10 successful logs locally when the Wi-Fi drops.
- Apply an auto-reconnection routine to flush all queued logs to your mock server (or webhook) once the connection stabilizes, and then clear the local memory
- emergency-override path that can bypass normal PIN validation.

3. System Architecture Overview

The system is organized into two cooperating layers that run inside a single Arduino loop() without the use of RTOS tasks or blocking delays:

3.1 The Access-Control Layer (Finite State Machine)

It is implemented as an explicit finite state machine (FSM) with five states, described in detail in Section 4.

It is unaware of networking entirely –

it only calls logAccessEvent(), whenever an access decision is made.

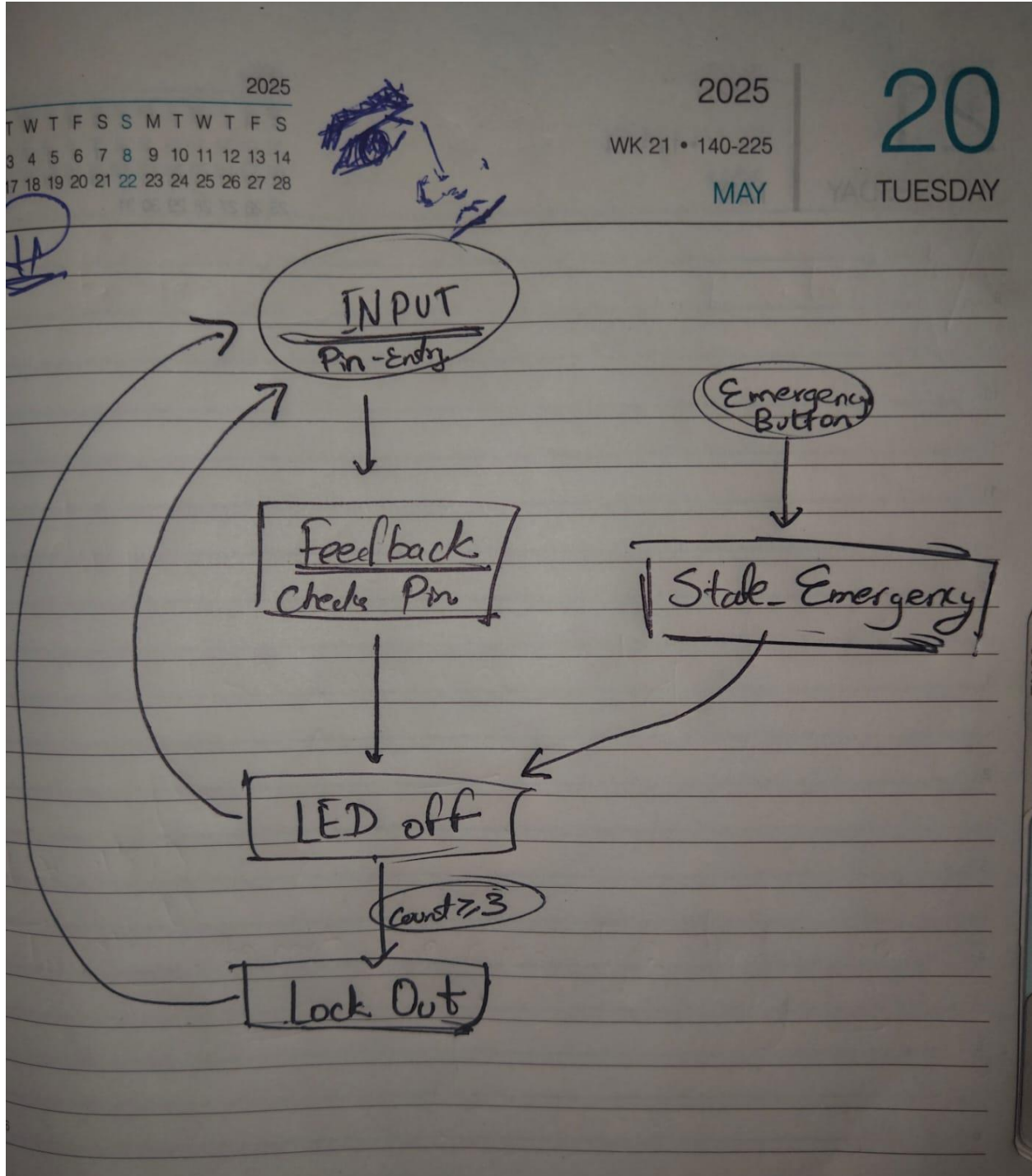
3.2 The Connectivity Layer (Cloud Sync)

This part handles everything network-related: connecting to Wi-Fi, syncing the clock over NTP, building the JSON payload, sending it over HTTP, and managing the offline queue

maintainWiFi(), attemptTimeSync(), and handleQueueFlush() — that run every single loop cycle.

4. State Machine Design

The terminal behaves as a five-state finite state machine at all times. At any given moment, exactly one state handler function is executing, which is controlled by `currentState` variable inspected once per loop().



4.1 State Descriptions

- STATE_INPUT:

Collects keypad digits one at a time via `readKey()`.

A 7-second inactivity timeout resets a partially-typed PIN.

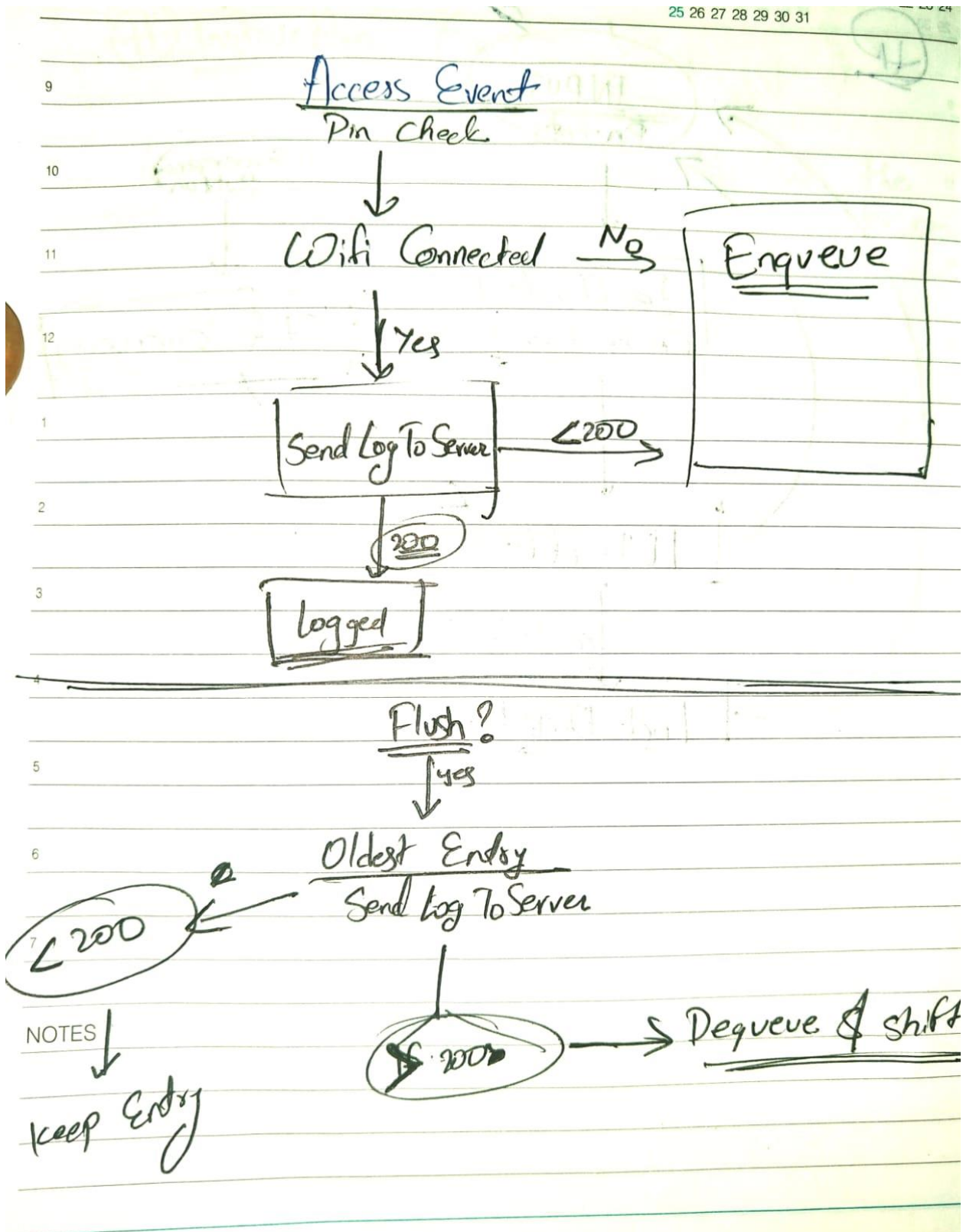
Transitions to `STATE_FEEDBACK` once the 4th digit is entered.

- `STATE_FEEDBACK`:
`check()` scans the local `userDatabase[]` for a matching PIN, updates the LCD, dispatches the log event, and unconditionally transitions to `STATE_LEDOFF`.
- `STATE_LEDOFF`:
 holds the result on-screen for 3 seconds (`ledOFF()`).
 if 3 consecutive wrong pins have accumulated, `currentState` goes to `STATE_LOCKOUT`; otherwise it returns to `STATE_INPUT`.
- `STATE_LOCKOUT`:
 disables further input for 15 seconds (`lockOut()`)
 then resets the failure counter and returns to `STATE_INPUT`.
- `STATE_EMERGENCY`:
 debounced hardware interrupt on pin 9.
`ghusjaa()` forces both outputs active, logs an "EMERGENCY"

4.2 Why an Explicit FSM

- each state's code stays separate and easy to test on its own.
- lets the Wi-Fi supervisor, time sync, and queue flush run on every single cycle (no matter the `currentState`)

5. Software Modules -- Detailed Explanation



5.1 maintainWiFi()

Called on every loop() iteration.

If disconnected, it attempts reconnection at most once every 5 seconds (RECONNECT_INTERVAL) // uses millis based timer.

When isConnected is true and wifiWasConnected is false ,then only performs the more expensive resync work -- restarting configTime() and resetting the time-sync timer .

5.2 Time Synchronization -- attemptTimeSync() and getCurrentTimestamp()

Using getLocalTime(), which returns false if a sync genuinely has not succeeded.

timeSynced flag is only set true once a sync has been positively confirmed.

Every log entry carries its own timeSynced flag, captured at the moment the event happened,

5.3 Logging Entry Point -- logAccessEvent()

builds a LogEntry, prints the full record to the Serial Monitor right away

then either sends the entry immediately (if Wi-Fi is connected) or places it in the offline queue.

5.4 Server Communication -- sendLogToServer()

turns a LogEntry into a compact JSON payload using StaticJsonDocument

then sends that payload with an HTTP POST through HTTPClient

HTTP connection is always explicitly closed with http.end()

5.5 Offline Queue -- enqueueLog(), dequeueFront(), handleQueueFlush()

fixed-size array of LogEntry structs (capacity 10).

If the queue fills up, the oldest entry is dropped to make room for the newest.

handleQueueFlush() sends just one queued entry every 5 seconds

5.6 Emergency Override -- the debounced ISR and ghusjaa()

The panic-button interrupt handler debounces itself, using nothing but millis() comparisons.

preventing mechanical bounce from firing multiple emergency events off one press.

6. Conclusion

Overall, the system does what it set out to do. Access control still runs on a simple, predictable state machine — enter a PIN, get checked, get let in or turned away — while everything to do with connectivity sits off to the side, quietly doing its own thing. WiFi drops, NTP hasn't synced yet, the server's unreachable — none of that ever holds up someone standing at the keypad. Logs get timestamped when possible, flagged as unsynced when not, queued when offline, and flushed automatically once things come back online

.